StudyMate Course Notes: Foundations of Artificial Intelligence

Chapter 1: Introduction to AI and Machine Learning Paradigms

1.1 What is Artificial Intelligence?

Artificial Intelligence (AI) refers to the simulation of human intelligence in machines that are programmed to think, learn, and solve problems. Traditional AI relies on symbolic reasoning and deterministic rule-based systems. In contrast, modern AI predominantly leverages Machine Learning (ML), allowing computer algorithms to automatically improve their performance through experience and data without being explicitly programmed for every edge case.

1.2 The Three Main Machine Learning Paradigms

Machine learning is broadly categorized into three core paradigms:
? Supervised Learning: The model learns from labeled datasets comprising input-output pairs. Common tasks include classification (e.g., spam detection, medical diagnosis) and regression (e.g., house price prediction, temperature forecasting). Popular algorithms include Linear Regression, Support Vector Machines (SVM), and Random Forests.
? Unsupervised Learning: The model identifies inherent patterns, clusters, or representations in unlabeled data. Key tasks include clustering (K-Means, Hierarchical Clustering) and dimensionality reduction (Principal Component Analysis - PCA, t-SNE).
? Reinforcement Learning (RL): An autonomous agent learns optimal decision policies through trial-and-error interactions with a dynamic environment, receiving scalar rewards or penalties. Famous examples include AlphaGo and robotic manipulation.

1.3 Key Concepts: Overfitting and Generalization

Generalization is the capability of a trained machine learning model to accurately predict outcomes on previously unseen test data. Overfitting occurs when a model learns the noise and idiosyncratic details of the training data rather than underlying patterns, resulting in poor validation accuracy. Common regularization techniques to combat overfitting include L1 (Lasso) and L2 (Ridge) penalties, Dropout, Early Stopping, and Cross-Validation.

StudyMate Course Notes: NLP & Embeddings

Chapter 2: Text Representation and Vector Space Models

2.1 Text Preprocessing and Tokenization

Natural Language Processing (NLP) bridges computational linguistics and statistical learning to enable computers to comprehend natural human language. The initial phase of NLP workflows involves text preprocessing: tokenization (splitting text into words, subwords, or characters), normalization (lowercasing, unicode cleanup), stop-word removal, and punctuation filtering. Modern LLMs employ subword tokenization algorithms such as Byte-Pair Encoding (BPE) and WordPiece.

2.2 From Bag-of-Words to Dense Vector Embeddings

Early text representations like Bag-of-Words (BoW) and TF-IDF (Term Frequency-Inverse Document Frequency) generated high-dimensional, sparse vectors that lacked semantic understanding and failed to capture synonymy or contextual nuances.
Dense vector embeddings revolutionized NLP by mapping words, sentences, or document chunks into a continuous low-dimensional semantic space (e.g., 768 or 1536 dimensions). In this latent space, semantically similar concepts reside in close geometric proximity. For instance, the vector distance between 'King' and 'Queen' mirrors that of 'Man' and 'Woman'.

2.3 Measuring Similarity: Cosine Similarity & Inner Product

To quantify semantic relevance between two vectors A and B, vector search systems compute the Cosine Similarity, which measures the cosine of the angle between them:
Cosine_Similarity(A, B) = (A · B) / (||A|| * ||B||)
When embeddings are L2-normalized (magnitude equals 1), the cosine similarity is mathematically equivalent to the dot product (inner product), allowing lightning-fast computations using matrix operations in specialized vector indexes.

StudyMate Course Notes: LLMs and RAG Systems

Chapter 3: Architecture of Retrieval-Augmented Generation

3.1 Limitations of Standalone Large Language Models

Large Language Models (LLMs) such as Google Gemini and GPT architectures demonstrate remarkable fluency and generative reasoning. However, standalone LLMs suffer from critical limitations in academic and domain-specific settings:
1. Knowledge Cutoff: LLMs cannot access private, newly published, or institution-specific documents.
2. Hallucinations: LLMs may generate plausible-sounding but factually fabricated statements with high confidence.
3. Lack of Attribution: Standard generations do not cite verifiable source pages or paragraphs.

3.2 The RAG Solution: How RAG Works

Retrieval-Augmented Generation (RAG) overcomes these limitations by combining a fast information retrieval mechanism with a generative language model. The RAG pipeline operates in three distinct phases:
1. Ingestion Phase: Documents (e.g., textbook PDFs) are extracted, preprocessed, segmented into overlapping text chunks, and converted into dense vector embeddings using an embedding model (e.g., text-embedding-004).
2. Vector Indexing: Embeddings and their source metadata (page number, chunk ID) are stored in an approximate nearest neighbor index such as FAISS (Facebook AI Similarity Search).
3. Query & Synthesis Phase: When a student asks a question, the query is embedded into the same vector space. FAISS retrieves the Top-K most semantically relevant chunks. A prompt engineering template combines the student's query with the retrieved chunks as grounded context, directing the LLM to generate an accurate, citation-backed answer.

3.3 Prompt Engineering and Guardrails

Prompt engineering is the disciplined practice of formulating inputs to guide LLM behavior predictably. In RAG applications, prompt templates enforce strict boundaries: 'Answer the question using ONLY the provided context. If the answer cannot be determined from the context, state clearly that the document does not contain this information.' This eliminates hallucinations and guarantees academic integrity.

StudyMate Course Notes: Vector Search & Evaluation

Chapter 4: FAISS Vector Indexing and Evaluation Metrics

4.1 Inside FAISS (Facebook AI Similarity Search)

FAISS is an open-source library engineered by Meta AI for highly efficient similarity search and clustering of dense vectors. Unlike traditional relational databases (SQL) that index scalar keys using B-Trees, FAISS organizes high-dimensional vectors for fast nearest-neighbor lookups.
Key FAISS index types include:
? IndexFlatL2: Computes exact Euclidean distance across all vectors. Accurate but computationally intensive for massive datasets.
? IndexFlatIP: Computes exact Inner Product. When vectors are L2-normalized, it calculates exact Cosine Similarity.
? IndexIVFFlat: Inverted File index that partitions the vector space into Voronoi cells to enable approximate nearest neighbor (ANN) search at immense scale.

4.2 NLP Chunking Strategies

Text chunking is a critical NLP step in RAG. If chunks are too small (e.g., 50 words), contextual meaning is fragmented. If chunks are too large (e.g., 2000 words), retrieval precision drops and irrelevant noise dilutes the prompt. Optimal chunking employs a sliding window approach with chunk overlap (e.g., 500-1000 characters with 100-200 character overlap) to preserve semantic boundaries between sentences and paragraphs.

4.3 Evaluation Metrics: Precision, Recall, and F1-Score

Evaluating AI systems requires quantifiable metrics:
? Precision = True Positives / (True Positives + False Positives): Measures exactness.
? Recall = True Positives / (True Positives + False Negatives): Measures completeness.
? F1-Score = 2 * (Precision * Recall) / (Precision + Recall): The harmonic mean of Precision and Recall, especially valuable when classes are imbalanced.